

WORKSHOP

Lab Manual

From Notebook to Production

DevOps & MLOps for CS/SE Graduation Projects

INSTRUCTOR

Mujahed Al-Tahleh

mujahed@ieee.org · github.com/mtahle/workshop-labs

Lab	Session	Duration	Page
Lab 1 — Your First Container	Session 1	25 min	2
Lab 2 — ML Inference API	Session 2	30 min	6
Lab 3 — GitHub Actions CI/CD	Session 2	25 min	11

Prerequisites for all labs: Docker Desktop running · Git installed · Python 3.10+ · VS Code · GitHub account

LAB

1

Your First Container

Session 1 · 25 minutes

Lab Objective

This lab introduces Docker by containerizing a Python script. The objective is to understand the difference between running code on your host machine versus inside a Docker container, and to practice writing a Dockerfile from scratch.

By the end of this lab, students will be able to write a Dockerfile for a Python application, build a Docker image, run a container, and verify the environment isolation that Docker provides.

Prerequisites

- Docker Desktop is installed and running
- Command-line terminal open (PowerShell, Terminal, or VS Code integrated terminal)
- **git clone <https://github.com/mtahle/devops-workshop-labs>** Workshop repository cloned:
- Completed Module 3 slides on Docker fundamentals

Lab Scenario

You have a Python script that prints system information and the current date. Your task is to package this script inside a Docker container so that it produces the same output on any machine — regardless of the operating system, Python version, or installed packages.

This mirrors what you will do with your graduation project model: the container becomes the unit of deployment, and the Dockerfile becomes the reproducible recipe for your environment.

Provided Files

Navigate to the lab directory:

```
cd workshop-labs/lab1-first-container
```

DIRECTORY CONTENTS

lab1-first-container/

— app.py	← Python script (already written)
— requirements.txt	← Dependencies file (empty — stdlib only)
— Dockerfile	← INCOMPLETE — you fill this in
— Dockerfile.solution	← Check your work after completing

app.py contents (read this first):

```
import platform
import datetime

print("Hello from inside Docker!")
print(f"Running Python {platform.python_version()}")
print(f"Today is: {datetime.date.today()}")
```

 Lab Tasks

1

Read all provided files

Before writing any code, read each file in the lab directory. Understand what app.py does and why requirements.txt is empty. This step is mandatory — never write a Dockerfile without reading the application first.

2

Write the Dockerfile

Open Dockerfile in your editor. It is currently empty. Using the Docker slides as reference, write a complete Dockerfile with the following instructions in this exact order:

1. FROM — use the official Python 3.11-slim base image
2. WORKDIR — set the working directory inside the container to /app
3. COPY — copy requirements.txt into the container
4. RUN — install dependencies with `pip install -r requirements.txt`
5. COPY — copy the rest of your code into the container
6. CMD — run `python app.py` when the container starts

Key Question: Why do we copy requirements.txt before the rest of the code? Think about Docker caching and what changes more often — your dependencies or your code.

3

Build the Docker image

Run the following command from the lab directory:

```
docker build -t lab1-app .
```

Watch the output carefully. Each line corresponds to a layer in your Dockerfile. Note how long the pip install step takes — you will see this become nearly instant on the next build due to layer caching.

4

Run the container and verify output

Run the container with:

```
docker run lab1-app
```

EXPECTED OUTPUT

```
Hello from inside Docker!
Running Python 3.11.x
Today is: 2026-06-20
```

Drop  in chat when your container produces the expected output. If you see an error, copy the full error message and post it in the chat for group troubleshooting.

5

Bonus — Observe layer caching

Make a small change to app.py — for example, add a second print statement. Rebuild the image and observe the output:

```
docker build -t lab1-app .
```

Notice which steps say "CACHED" — the FROM, WORKDIR, and pip install layers are not re-executed because they have not changed. Only the COPY . . layer and below are rebuilt. This is why we structure Dockerfiles with stable layers first.

Compare your Dockerfile with Dockerfile.solution to check your work.

**Lab Deliverables**

- A complete, working Dockerfile that successfully builds and runs
- Confirmed terminal output matching the expected output above
- Bonus: screenshot or description of the layer caching behavior

**Evaluation Criteria**

Criterion	Weight	Self Check
Dockerfile uses the correct base image (python:3.11-slim)	20%	☐
Layer order is correct: requirements before code copy	20%	☐
Container builds without errors	20%	☐
Container runs and produces correct output	30%	☐
Bonus: student can explain layer caching benefit	10%	☐

**Expected Learning Outcomes**

- ✓ Understand what a Docker image and container are and how they differ
- ✓ Write a basic Dockerfile using FROM, WORKDIR, COPY, RUN, and CMD
- ✓ Understand Docker layer caching and why dependency installation comes before code copy
- ✓ Build and run a Docker container from the command line
- ✓ Recognise how containerisation eliminates "it works on my machine" problems

LAB

2

ML Inference API

Session 2 · 30 minutes

Lab Objective

This lab demonstrates the standard pattern for deploying a machine learning model as a REST API inside a Docker container. Students will build and run a FastAPI inference server for a scikit-learn Iris classifier, then interact with it through an auto-generated web UI.

By the end of this lab, students will understand how to package a trained model, expose it as an HTTP endpoint, and test predictions — the same pattern applies to any ML model in any framework (TensorFlow, PyTorch, XGBoost, or custom models).

Prerequisites

- Lab 1 completed — Docker build and run commands are familiar
- Module 3 (continued) slides reviewed — FastAPI and ML-in-Docker pattern
- **workshop-labs/lab2-ml-api** Terminal positioned in:
- No additional installations required — everything runs inside the container

Background: The ML Deployment Pattern

There are three components to a containerised ML API:

model.py — Trains the model and saves it as a .pkl file. This runs at image build time.

main.py — FastAPI application that loads the saved model and exposes a /predict endpoint.

Dockerfile — Builds both into a single image. Model is trained during the build, not at startup.

Training at build time means users never wait for model initialisation on first request. The model is always ready when the container starts.

Lab Scenario

You are deploying an Iris species classifier as an internal API. Botanists at a research institute will submit four flower measurements and receive a species prediction. The model must be reproducible, containerised, and accessible via an HTTP interface.

The Iris dataset classifies flowers into three species based on sepal and petal measurements. While simple, this deployment pattern scales directly to complex models in your graduation project.

Provided Files

Navigate to the lab directory:

```
cd workshop-labs/lab2-ml-api
```

DIRECTORY CONTENTS

lab2-ml-api/

├── model.py

← Trains Iris classifier, saves model.pkl

```

├── main.py           ← FastAPI app with /predict endpoint
├── requirements.txt  ← Pinned: fastapi, uvicorn, sklearn, numpy
├── Dockerfile        ← Complete – read before building
└── tests/
    └── test_model.py ← 5 model validation tests

```

Key code to understand before building:

The /predict endpoint in main.py:

```

class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict(data: IrisInput):
    features = np.array([[data.sepal_length, data.sepal_width,
                          data.petal_length, data.petal_width]])
    prediction = model.predict(features)
    species = ["setosa", "versicolor", "virginica"]
    return {"prediction": species[prediction[0]]}

```

The Dockerfile (key difference from Lab 1):

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY model.py .
RUN python model.py          # ← trains model at BUILD time
COPY main.py .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

**Lab Tasks****1****Read all files and understand the architecture**

Read model.py, main.py, and the Dockerfile completely before running any command.
Answer these questions to yourself before proceeding:

- What does model.py produce? Where is the output saved?
- What does BaseModel in main.py give us automatically?
- Why does the Dockerfile run python model.py before copying main.py?

2**Build the Docker image**

From the lab2-ml-api directory, build the image:

```
docker build -t iris-classifier .
```

During the build, look for the output from model.py:

EXPECTED OUTPUT

Model trained and saved.

This confirms the model was trained at build time and is embedded in the image. Any container started from this image has the model ready immediately.

Run the API server

Start the container and map port 8000:

```
docker run -p 8000:8000 iris-classifier
```

3

You should see uvicorn starting up. Leave this terminal open — the server runs in the foreground. Open a new terminal window for the next steps.

EXPECTED OUTPUT

```
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000
```

Test predictions via the /docs UI

Open your browser and navigate to:

```
http://localhost:8000/docs
```

FastAPI automatically generates an interactive Swagger UI. Follow these steps:

7. Click POST /predict, then click "Try it out"
8. Enter the following JSON in the request body:

4

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

9. Click Execute and verify the response:

EXPECTED OUTPUT

```
{"prediction": "setosa"}
```

5**Try all three species — bonus challenge**

Use the /docs UI to test the following inputs and confirm you can get all three species:

sepal_length	sepal_width	petal_length	petal_width	Expected Species
5.1	3.5	1.4	0.2	setosa
6.3	3.3	4.7	1.6	versicolor
6.7	3.0	5.2	2.3	virginica

Share your results in the chat. Try to find your own input combination that produces each species.



Lab Deliverables

- Docker image iris-classifier builds successfully
- API server runs and responds to POST /predict requests
- All three species confirmed through the /docs UI
- Bonus: student can explain why training at build time is preferable to training at startup



Evaluation Criteria

Criterion	Weight	Self Check
Image builds without errors, including successful model training	25%	☐
Container starts and uvicorn is accessible on port 8000	20%	☐
POST /predict returns correct predictions for provided inputs	30%	☐
Student can explain the role of BaseModel in input validation	15%	☐
Bonus: all three species tested with correct results	10%	☐



Expected Learning Outcomes

- ✓ Understand the three-component pattern: trainer + API server + Dockerfile
- ✓ Expose a trained ML model as a REST API using FastAPI
- ✓ Use Pydantic BaseModel for automatic request validation
- ✓ Access and test an API using Swagger UI at /docs
- ✓ Understand why model training belongs in the build phase, not the startup phase
- ✓ Apply this containerisation pattern to any ML framework or model type

LAB

3

GitHub Actions CI/CD Pipeline

Session 2 · 25 minutes

Lab Objective

This lab implements a Continuous Integration pipeline using GitHub Actions. Every time code is pushed to the main branch, the pipeline automatically installs dependencies, trains the model, runs tests, and builds the Docker image — providing instant feedback on whether the change breaks anything.

By the end of this lab, students will have a working CI pipeline that turns every git push into an automated quality gate. This is the foundation of professional ML engineering workflows.

Prerequisites

- Lab 2 completed — the iris-classifier repo exists and runs locally
- GitHub account with the workshop repository forked or the Lab 2 code pushed
- Module 4 slides reviewed — CI/CD concepts and GitHub Actions workflow syntax
- Terminal open in the lab2-ml-api directory (or your fork of it)

Background: What CI/CD Gives You

Without CI/CD

git push → manually run tests → manually rebuild Docker image → manually deploy → wait and hope nothing broke

With GitHub Actions

git push → pipeline triggers automatically → tests run → image builds → you get a green or red badge in seconds

Free tier: GitHub provides 2,000 Actions minutes per month for public repositories. This lab uses approximately 2–3 minutes per push.

Lab Scenario

The Iris classifier API from Lab 2 is in production. A team member changes the preprocessing logic. How do you know the model still works before deploying? You set up a CI pipeline that automatically validates every push. The pipeline acts as a guardian: no broken code reaches production.

In this lab you will write the pipeline configuration file, push it to GitHub, watch it run in the Actions tab, and then deliberately break it to see the failure feedback — because understanding failure is as important as success.

Provided Files

Navigate to the lab directory:

```
cd workshop-labs/lab3-github-actions
```

DIRECTORY CONTENTS

lab3-github-actions/

```
└─ ci.yml.template           ← Copy this to .github/workflows/ci.yml
└─ ci-with-docker-push.yml.template ← Bonus: pushes image to Docker Hub
```

The workflow file (ci.yml.template):

```
name: ML CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test-and-build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v4
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Train model
        run: python model.py

      - name: Run tests
        run: pytest tests/ -v

      - name: Build Docker image
        run: docker build -t iris-classifier .
```



Lab Tasks

Create the workflow file

Create the required directory structure and copy the template:

```
# From your repo root (the directory containing lab2-ml-api files):
mkdir -p .github/workflows
cp lab3-github-actions/ci.yml.template .github/workflows/ci.yml
```

1

Open `.github/workflows/ci.yml` in your editor and read it line by line. Answer these questions:

- What event triggers this workflow?
- Which operating system does the runner use?
- What does `actions/checkout@v4` do?
- What is the order of steps? Why does `pytest` run before `docker build`?

2

Push to GitHub and watch the pipeline run

Stage, commit, and push the workflow file:

```
git add .github/
git commit -m "Add GitHub Actions CI pipeline"
git push origin main
```

Then navigate to your GitHub repository and click the Actions tab. You will see the workflow running in real time. Click on the running workflow to see each step execute.

EXPECTED OUTPUT

```
ML CI Pipeline ✓ Set up Python 3.11
                ✓ Install dependencies
                ✓ Train model
                ✓ Run tests
                ✓ Build Docker image
```

Post a screenshot of your green Actions run in the chat.

3

Break the pipeline on purpose

This step is mandatory — understanding how failures appear is as important as making things work.

Open tests/test_model.py and find the test that checks the setosa prediction. Change the expected class from 0 to 1 (which is wrong):

```
# BEFORE (correct):
assert model.predict(features)[0] == 0

# AFTER (deliberately wrong):
assert model.predict(features)[0] == 1
```

Commit and push this change:

```
git add tests/
git commit -m "Test: intentionally break CI"
git push origin main
```

Go to the Actions tab and watch the pipeline fail on the "Run tests" step. Notice: the Docker image is never built when tests fail. This prevents broken code from being packaged.

4

Fix the pipeline and restore green status

Revert the test back to the correct assertion:

```
assert model.predict(features)[0] == 0 # correct - setosa is class 0
```

Commit and push:

```
git add tests/
git commit -m "Fix: restore correct test assertion"
git push origin main
```

Confirm the pipeline returns to green. You have now completed the full CI feedback loop: push → fail → fix → green.

5

Bonus — Add Docker Hub push step

If you have a Docker Hub account, use `ci-with-docker-push.yml.template` to extend the pipeline so it automatically pushes the image to Docker Hub on every successful main branch push.

You will need to add two GitHub repository secrets: `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`. Go to repository Settings → Secrets and variables → Actions to add them.



Lab Deliverables

- A `.github/workflows/ci.yml` file committed in your GitHub repository
- A successful (green) GitHub Actions run visible in the Actions tab
- A failed run showing the deliberately broken test (screenshot)
- Restored green run after fixing the test



Evaluation Criteria

Criterion	Weight	Self Check
Workflow file is correctly structured and triggers on push to main	20%	☐
All five pipeline steps complete successfully (green run)	30%	☐
Student can explain what each step in the pipeline does	20%	☐
Intentional failure observed and documented	20%	☐
Bonus: Docker Hub push step configured and working	10%	☐



Expected Learning Outcomes

- ✓ Write a GitHub Actions workflow file using YAML syntax
- ✓ Understand the concepts of triggers, jobs, runners, and steps
- ✓ Implement a CI pipeline that tests and builds an ML project automatically
- ✓ Observe and interpret pipeline failures and fix them quickly
- ✓ Understand why test failures block the Docker build step
- ✓ Explain how this pipeline extends to continuous delivery (push to registry on success)